

ECS 160 – Reflections ctd.

Instructor: Tapti Palit

Teaching Assistant: **Xingming Xu**



Reflections

- Last time we discussed the importance of reflection
- Discovering classes, methods, fields at runtime
- Dynamically calling and modifying behavior
- Turns code into data the program can inspect



Key terms

- Introspection
 - Observe metadata
- Intercession
 - Modify behaviour



java.lang.reflect

- Class <?>
- Field
- Method
- Constructor



Reflection Stack

- JVM runtime model
- Reflection API
- High-level frameworks of your choice
- Bytecode in program files



Purpose

- Adapt!
- Adapt to unknown types at runtime
- External configurations
- Greater user customization
- Provide dynamic environments



Flexible systems

- We cannot predict every scenario and context at runtime
- Software:
 - No longer runs on just one machine + environment
 - Will change after delivery
 - Might have to accept new classes (microservices, libraries, plugins etc.) after deployment
- Data evolves
- Enables orchestration

Contrasting Java Reflection

- Java was initially designed as
 - Statically typed
 - Compiled ahead of time
 - Strictly encapsulated (private fields)
- Reflection is a whole ceremony
 - Explicitly ask to break encapsulation and retrieve metadata
- Everything is static,
 - Until we need dynamism



JVM

- JVM preserves
 - Class names
 - Methods, fields
 - Types, signatures
 - Annotations, modifiers



Class

- `Class<?> c = obj.getClass();`
- We can get fields, methods, annotations, signatures, etc.
- ```
for (Method m : c.getDeclaredMethods()) {
 System.out.println(m.getName());
}
```
- This is the code reading itself!



# Class is not class

- Class isn't the exact same thing as
  - `class Car { ... }, Car c = new Car();`
- `Class <?> c = c.getClass();`
  - Will represent Car at runtime
- Generated at JVM, describes behavior of your class to the runtime

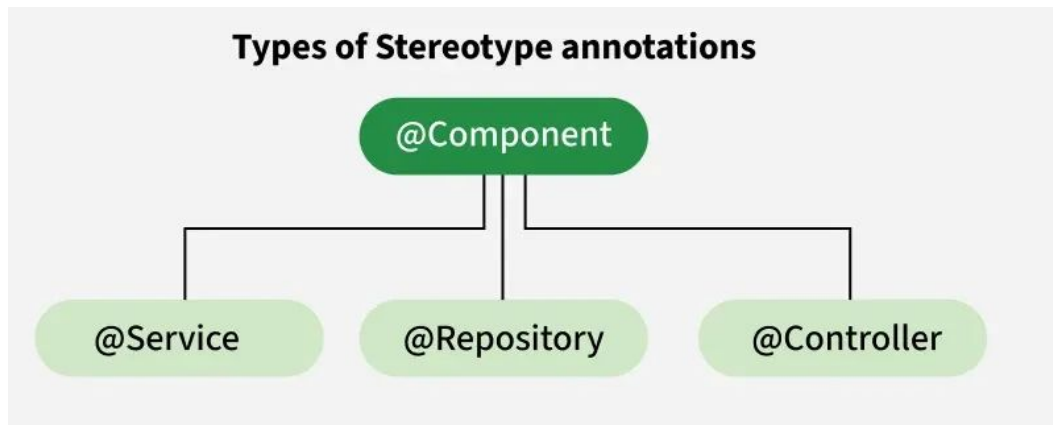
# Dependency Injection

- Spring scans:
  - Types, annotations, constructors
- Constructs dependency graphs



# Dependency Graph Construction

- As an example...
- Scans class path for @Component and other “stereotype” annotations
- Detect, auto-register, and manage these beans



# Spring Reflection

- Finds classes
- Stores metadata
- Registers them as candidates in container



# Dependency Map

- Examines each bean
  - Constructor parameters
  - Annotated fields
  - Setters with dependencies
- Detects relationships to construct a graph



# Constructor Injection

- Spring gets to call the constructor
- In complex, non-circular cases
  - Looks in graph to find primary objects
  - Looks for @Autowiring order
- @Autowiring is ultimately a topological ordering
  - Build things with no dependencies
  - Resolves needed dependencies per node/object
  - Instantiates it
  - Wires fields, *method injection*



# Other Wrappers

- Caching
- Security
- Async behavior
- Spring *intercepts* method calls to this object, creating a proxy around it to manage these issues



# Enabling Orchestration via Spring

- Scan class path
- Find beans (objects)
- Creates objects
- Injects fields/constructors
- Wrap with proxies



# End Result

- For us, no boilerplate,
  - Add features *without changing code*
- For Spring,
  - Scanned each class
  - Built a dependency graph
  - Injected dependencies
- This is runtime architecture



# Questions

- An example of dependency injection

